

Docker: integrazioni e Compose

Marco Alberti (lbrmrc@unife.it)

Professore Associato di Informatica

Dipartimento di Matematica e Informatica

Università degli Studi di Ferrara

Volumi: accesso al filesystem locale da un container

opzione `-v` (`--volume`) del comando `container run` : monta una cartella locale nel filesystem del container

Esempio: `-v $(pwd) : /home/host` monta la cartella corrente nella cartella `/home/host` del container

Attività: modifica di file locali da un container

1. Creare ed eseguire un container, basato su Ubuntu, montando in esso una cartella locale
 - prima fate un backup!
2. Modificare il contenuto della cartella da dentro il container (ad esempio editando o copiando un file)
3. Chiudere il container
4. Verificare che il contenuto della cartella locale è cambiato

Attività: compilazione di un progetto locale da un container

1. Creare una nuova cartella
2. Creare un semplice programma C++ (va bene anche un Hello World)
3. Montare la cartella in un container dotato di compilatore C++
4. Compilare il programma nel container
5. Eseguire il programma nel container
6. Chiudere il container
7. Eseguire il programma nell'host

Dockerfile multistage

E' possibile creare immagini **temporanee** e copiare file creati da un'immagine a un'altra. Esempio:

```
FROM golang:1.16 AS builder
WORKDIR /go/src/github.com/alexellis/href-counter/
RUN go get -d -v golang.org/x/net/html
COPY app.go ./
RUN CGO_ENABLED=0 go build -a -installsuffix cgo -o app .

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /go/src/github.com/alexellis/href-counter/app ./
CMD [ "./app" ]
```

Attività: build e run

Usando un Dockerfile multistage, fare il build di un progetto usando un'immagine attrezzata allo scopo, dopo di che creare una seconda immagine che, lanciata, lancia l'eseguibile creato allo stage precedente

Workflow: integrazione con Visual Studio Code

Estensione `ms-vscode-remote.remote-containers` (Dev Containers)

Si crea nella radice della cartella dei sorgenti un Dockerfile che definisce l'ambiente di sviluppo

Aperto la cartella, verrà proposto di riaprirla nel container definito dal Dockerfile

In alternativa, usare il comando **open folder in container**

Volendo, utilizzando il file `.devcontainer/.devcontainer.json` si può ulteriormente personalizzare l'ambiente (ad esempio installando l'estensione C/C++)

Insomma è possibile ricreare a partire da un repository ridotto tutto l'ambiente di sviluppo in modo automatico

Docker Compose

E' un plugin di Docker che consente di gestire l'esecuzione coordinata di più container
Ad esempio è utile nelle architetture a microservizi:

- avvio il server di database
- parallelamente
 - avvio il servizio di autenticazione
 - avvio il servizio di API
- avvio il server web

Infrastructure as code: il coordinamento fra i vari container è definito da un file di configurazione (`docker-compose.yaml`).

Parentesi: YAML

Linguaggio usato per definire pipeline in Gitlab (e per Docker Compose)

```
oggetto1:
  attributo1: valore1
  attributo2:
    sottoattributo1: sottovalore1
    sottoattributo2: sottovalore2
  attributo_lista:
    - elemento1
    - elemento2:
        attributo1: valore1
        attributo2: valore2
    - attributo1: valore1
      attributo2: valore2
oggetto2: valore2
```

Configurazione Docker Compose

File YAML di nome `docker-compose.yaml` . Esempio:

```
version: "3.9"
services:
  web:
    build: .
    ports:
      - "8000:5000"
  redis:
    image: "redis:alpine"
```

Documentazione

<https://docs.docker.com/compose/>

Servizio

Corrisponderà a un container.

Si può definire con l'attributo `image`

```
services:  
  servizio:  
    image: debian
```

oppure con l'attributo `build` se la cartella indicata contiene un `Dockerfile`

```
services:  
  servizio:  
    build: .
```

In questo modo l'immagine verrà ricreata alla creazione del container.

Esecuzione comandi

E' possibile impostare comandi da eseguire con l'attributo `command`

```
services:  
  servizio:  
    image: ubuntu  
    command: /bin/bash
```

Se il comando è composto da più token usare una lista:

```
command:  
- ls  
- /usr/share
```

Per eseguire un servizio (container) eliminandolo al termine:

```
docker compose run --rm servizio
```

Attività

1. Definire in un Dockerfile un'immagine basata su Ubuntu con Midnight Commander
2. Definire con Docker Compose un servizio che utilizzi il Dockerfile ed esegua Midnight Commander
3. Eseguire il servizio

Docker Compose e linea di comando

In generale, c'è una corrispondenza fra attributi di Docker Compose e opzioni della linea di comando.

Esempio volumi:

l'opzione `-v cartella_locale:cartella_container`
corrisponde all'attributo

```
volumes:  
- cartella_locale:cartella_container
```

Attività

1. Creare un file Docker Compose che monti la cartella corrente in un container basato su immagine Ubuntu
2. Eseguire il servizio
3. Constatare che la cartella indicata contiene il file `docker-compose.yaml`

Più servizi

- L'uso più comune di Docker Compose è una forma semplice di orchestrazione, cioè l'esecuzione coordinata di più container.
- L'attributo `services` può definire più servizi
- Il comando `docker compose up` fa partire tutti i servizi

Attività

1. Definire con Docker Compose due servizi ognuno dei quali stampa un diverso messaggio
2. Eseguire i due servizi

Comandi principali di controllo

- `up` : creare e avviare tutti i container della specifica
- `down` : fermare ed eliminare tutti i container
- `start` : avviare tutti i container (che erano stati creati e fermati)
- `stop` : fermare tutti i container
- `run` : eseguire uno dei services (container)

Dipendenze

A volte è necessario indicare che un servizio dipende da un altro (ad esempio server web dipende dal server di database)

Attributo

```
depends_on:  
- <servizio>
```

Se è necessario assicurarsi che sia terminato:

```
depends_on:  
  <servizio>  
  condition: service_completed_successfully
```

Vedere anche la condizione `service_healthy`

Attività

1. Dato il sorgente di un progetto, creare la configurazione necessaria per due servizi:
 - uno di build
 - uno di esecuzione, dipendente dal completamento del primo
2. Eseguire il tutto con il comando `docker compose up`

MySQL e PhpMyAdmin da linea di comando

```
docker network create mynet
```

```
docker container run --name db -e MYSQL_ROOT_PASSWORD=my_secret_password -  
e MYSQL_DATABASE=app_db -e MYSQL_USER=db_user -e  
MYSQL_PASSWORD=db_user_pass -e MYSQL_TCP_PORT=6033 -p 6033:3306 -v  
dbdata:/var/lib/mysql --network mynet mysql
```

```
docker container run --name pma -e PMA_HOST=db -e PMA_PORT=6033 -e  
PMA_ARBITRARY=1 -p 8081:80 --network mynet phpmyadmin/phpmyadmin
```

Il servizio è accessibile a `localhost:8081` con

- host: `db`
- username: `db_user`
- password: `db_user_pass`

File Docker Compose corrispondente

```
version: '3'

networks:
  mynet:

services:
  db:
    image: mysql:latest
    container_name: db
    environment:
      MYSQL_ROOT_PASSWORD: my_secret_password
      MYSQL_DATABASE: app_db
      MYSQL_USER: db_user
      MYSQL_PASSWORD: db_user_pass
      MYSQL_TCP_PORT: 6033
    ports:
      - 6033:3306
    volumes:
      - dbdata:/var/lib/mysql
    networks:
      - mynet
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: pma
    depends_on:
      - db
    environment:
      PMA_HOST: db
      PMA_PORT: 6033
      PMA_ARBITRARY: 1
    restart: always
    ports:
      - 8081:80
    networks:
      - mynet
volumes:
  dbdata:
```

Opzioni linea comando e attributi

- rete: `-n mynet`

```
networks:  
  - mynet
```

- variabili d'ambiente: `-e <var1>=<val1> -e <var2>=<val2>`

```
environment:  
  var1: val1  
  var2: val2
```

- volumi: `-v locale:container` (vedere anche `volumes toplevel` nell'esempio)

```
volumes:  
  - locale:container
```


Attività

Il programma Python alla diapositiva seguente usa il modulo `mysql-connector-python` per recuperare dati da un database MySQL e stamparli

Per installare un modulo Python:

```
pip install <nome_modulo>
```

Definire con Docker Compose due container:

- uno per MySQL (simile all'esempio precedente)
- uno per l'applicazione Python (usando un Dockerfile basato sull'immagine `python`)

Eseguire l'applicazione.

```
import mysql.connector
import os
import time

time.sleep(5)

mydb = mysql.connector.connect(
    host=os.environ["MYSQL_SERVER"],
    user=os.environ["MYSQL_USER"],
    password=os.environ["MYSQL_PASSWORD"]
)

cursor = mydb.cursor()
databases = ("show databases")
cursor.execute(databases)
for (db) in cursor:
    print (db[0])
```

Segreti: alternativa a variabili di ambiente

<https://docs.docker.com/compose/use-secrets/>

Limitazioni di Docker Compose come strumento di orchestrazione (senza Swarm)

- Tutti i container sullo stesso container engine

Per installazioni più piccole, la semplicità di Compose può essere un fattore decisivo.

Alcune funzionalità di sistemi di orchestrazione più sofisticati:

- `restart`
- `replicas`

Altro

`docker compose build`

`working_dir`

`command: bash -c "`

`comando1 &&`

`comando2`

`"`